Video Voice Translator

1. Project Background

Cross-language video localization is still expensive and slow for individual creators, educators, and small media teams. A typical workflow requires subtitle generation, manual translation, voice-over recording, audio alignment, and final video rendering. This project reduces that friction by turning the full workflow into an end-to-end AI-assisted pipeline.

Video Voice Translator is a multimodal content creation tool built for Track 1 of the 2026 AMD AI DevMaster Hackathon. It focuses on practical video dubbing and content repurposing: a user uploads a video, selects a target language, and receives a translated dubbed output while preserving the original speaking style as closely as possible.

2. Target Users and Application Scenarios

Target Users

- Content creators who want to localize short videos quickly
- Educators who need bilingual teaching materials
- Small media teams producing English and Chinese versions of the same content
- Product teams preparing multilingual demos for marketing or developer education

Application Scenarios

- Short-form social media video localization
- Technical tutorial dubbing
- Course content adaptation between English and Chinese
- Demo video preparation for product launches or developer showcases

3. System Architecture

The system uses a split frontend/backend architecture with a dedicated worker for long-running media jobs.

Architecture Components

- **Frontend**: React + TypeScript + Vite
- **Backend API**: FastAPI
- **Background execution**: worker process for translation tasks
- **Media processing**: FFmpeg-based extraction and synthesis
- **Persistent task state**: JSON task records under ‘data/task_states/‘
- **Output storage**: generated task outputs under ‘data/outputs/‘

Execution Flow

1. User uploads a video or audio file in the Web UI
2. Backend registers the task and stores upload metadata
3. Worker executes the nine-stage translation pipeline
4. Progress and status are exposed through the API
5. Final media and task artifacts are stored on disk
6. Completed jobs can be reopened later through the history panel

4. Model and Algorithm Introduction

The full pipeline contains the following AI and media processing stages:

4.1 Audio Extraction and Normalization

The input media is normalized into a consistent audio representation using FFmpeg. This step ensures later stages receive a predictable sample rate and channel layout.

4.2 Vocal Separation

The system separates vocals and background audio to improve speech recognition quality and to preserve ambient content for the final dubbed output.

4.3 Speech Recognition

Speech is transcribed with Whisper-family models. This branch defaults to the native ‘whisper‘ backend because it is more suitable for the AMD/ROCm environment used in this project. Timestamped segments are generated for alignment.

4.4 Text Translation

The recognized transcript is translated in batches via **DeepSeek-V4-Flash** served from the AMD Radeon developer TokenFactory endpoint (‘https://developer.amd.com.cn/radeon/tokenfactory‘). The translation layer uses an OpenAI-compatible client with retry logic and response validation to improve robustness when segment-level responses are inconsistent.

4.5 Voice Cloning

Translated segments are synthesized with IndexTTS2. Reference audio is extracted from the source material so the output keeps a similar speaker identity and speaking style.

4.6 Audio Merging and Video Rendering

The synthesized clips are aligned back to the original timeline, merged with preserved background audio, and rendered into the final translated video.

5. AMD Radeon / ROCm Adaptation

This project was adjusted specifically for AMD GPU execution:

- The default ASR backend is native ‘whisper‘ instead of ‘faster-whisper‘, which reduces compatibility issues on ROCm in this branch.
- ‘IndexTTS2‘ uses FP16 for performance, while disabling CUDA-only kernels that are not appropriate for ROCm.
- The installation scripts attempt to prepare ROCm-compatible PyTorch automatically on AMD cloud machines.
- The system isolates long-running translation tasks into a worker process to improve runtime stability during demos.
- Progress and result persistence reduce the chance of losing demo visibility after a frontend refresh or server restart.

6. Core Features and User Experience

- Web-based upload and translation workflow
- Support for English and Chinese translation directions
- Translation history with reopen and delete actions
- Progress updates across major pipeline stages
- Final result replay in the browser

- Command-line entry point for non-UI evaluation

7. Demo and Reproducibility Design

The repository contains startup and management scripts intended to reduce evaluation friction:

- ‘./install_all.sh‘ for one-click environment setup
- ‘./service.sh up‘ for frontend/backend startup
- ‘./service.sh status‘, ‘restart‘, ‘logs‘, and ‘down‘ for demo control
- ‘data/demo/demo.mp4‘ as a ready-to-review demonstration artifact

The recommended judging path is:

1. watch the included demo video,
2. start the split web stack,
3. upload a short English video,
4. run translation to Chinese,
5. review the final dubbed result and history replay.

8. Innovation and Practical Value

This project focuses on practical multimodal creation rather than a toy demo. The main value lies in integrating several AI capabilities into a single creator-facing workflow:

- speech recognition,
- translation,
- voice cloning,
- synchronized audio reconstruction,
- and deliverable video rendering.

The work is especially relevant for bilingual content creation and developer education, where the ability to turn one source video into multiple localized versions has direct productivity value.

9. Current Limitations

- The best current path is short-form, single-speaker content.
- Very long uploads may require proxy timeout tuning in some deployment setups.
- Multi-speaker support exists but is not the strongest demonstration path in the current branch.

10. Conclusion

Video Voice Translator demonstrates a complete multimodal content creation workflow accelerated by an AMD-oriented software stack. It converts raw media into a translated and dubbed output through a reproducible pipeline, a usable Web UI, and a practical set of scripts for setup and judging. The project is aimed at making multilingual video creation faster, more accessible, and easier to demonstrate on AMD Radeon GPU environments.